

CS409m: Introduction to Cryptography

Lecture 08 — Encrypting Multiple Messages, and Stretching a PRG via the Hybrid Argument

Compiled study notes, expanded from the handwritten proof notes
and the lecture slides (Instructor: Chethan Kamath)

Based on lecture materials dated 21–27 Aug 2026

Abstract

These notes merge and expand on *two* source documents for Lecture 08 of CS409m. The *slide deck* sets up the lecture’s real goal — extending single-message PRG-based encryption (Lecture 07) to **multiple messages**, which forces us to confront how to **stretch** a PRG’s output to arbitrary polynomial length while preserving pseudorandomness, introducing the powerful **hybrid argument** proof technique along the way (illustrated with a memorable Coca-Cola/Campa-Cola analogy). The *handwritten notes* then supply the complete, rigorous proof of **Theorem 4** (the PRG length-extension theorem) that the slides only sketch informally — formalizing the $2n+1$ hybrid worlds $H_0, \dots, H_{2n}$ and the reduction distinguisher D built from any distinguisher D' for the stretched PRG. Every diagram from both PDFs is redrawn in T_EX here, and additional explanatory material is clearly marked with a ★ box.

Contents

1	Recap from the Previous Two Lectures	3
1.1	(Catch-Up) PRG #2 and PRG #3, Recapped	3
2	Today’s Goal: Encrypting <i>Multiple</i> Messages	4
2.1	Setting and the Naive Construction	4
2.2	Formalizing Multi-Message Security	5
3	How to Stretch a PRG	7
4	Proving Pseudorandomness: The Hybrid Argument	8
4.1	A Warm-Up: The Coca-Cola / Campa-Cola Analogy	8
4.2	What Is “Coca/Campa Cola” in the PRG Case?	8
4.3	Theorem 4, and the Two-Hybrid Warm-Up	8
5	The Full Formal Proof of Theorem 4	10
5.1	Setting Up the $2n + 1$ Hybrid Worlds	10
5.2	From Adjacent-Hybrid Distinguishing to a PRG Distinguisher	11
5.3	Analysis	12

6	Taking Stock: The Cost of a Hybrid Argument	14
6.1	Why Loss in Security Matters	15
7	Recap and Next Lecture	16

1 Recap from the Previous Two Lectures

- **Task:** secret communication of *one long message* with shared keys.
- **Threat model:** computational secrecy against eavesdroppers (recall the attack-model/security-goal breakdown and Kerckhoffs’s principle from Lecture 07).
- **Computational OTP (COTP):** the one-time pad, except the key is first expanded/stretched using a PRG G .
- **PRG constructions** seen so far: *practical*, via stream ciphers (LFSR, Trivium, Salsa/ChaCha); *theoretical*, via hard problems (subset-sum, factoring).

Theorem 1.1 (Theorem 1, Lecture 06, rephrased). *If PRGs exist, then so do computationally-secret SKEs.*

★ Added by Claude — Same theorem, sharper phrasing

This is exactly the Theorem 1 whose full reduction proof was reproduced in the Lecture 07 notes (“if G is a PRG, then Construction 2/COTP is computationally secret against eavesdroppers”), just restated existentially: it says that the mere *existence* of PRGs is enough to bootstrap the existence of secure encryption for a single message. This existential phrasing — “if primitive X exists, then so does primitive Y ” — is the standard way complexity-based cryptography states its results, since it cleanly separates *what we can build from what* from *whether the underlying assumption is actually true*.

1.1 (Catch-Up) PRG #2 and PRG #3, Recapped

The slides briefly re-recap the two “theoretical” PRG constructions from Lecture 07, since they underpin later exercises. We restate them tersely here (see the Lecture 07 notes for the full discussion, including the Merkle–Hellman and Blum–Blum–Shub context boxes).

PRG #2 (Subset-Sum). For a prime modulus q and $a_1, \dots, a_n < q$ sampled at random (with $q > 2^n$), assumed hard to solve $\sum_{i \in \mathcal{I}} a_i \equiv 0 \pmod{q}$:

$$G_{q,a_1,\dots,a_n}(s_1 \| \dots \| s_n) := \sum_{i=1}^n s_i a_i \pmod{q}.$$

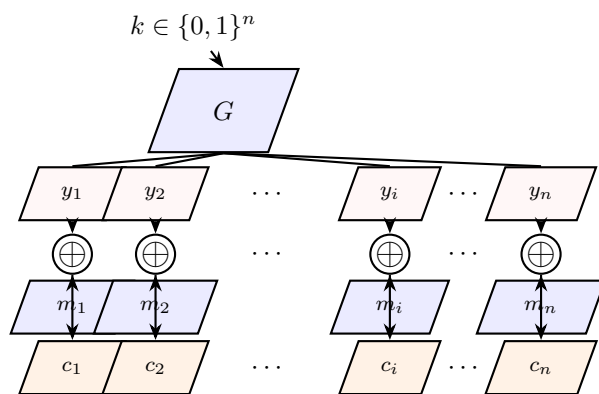
PRG #3 (Factoring-based, i.e. Blum–Blum–Shub). For $N = q_1 q_2$ a product of two random same-length primes, assumed hard to factor: Init samples $x_0 \leftarrow [0, N-1]$; Eval on x_i computes $x_{i+1} := x_i^2 \bmod N$ and outputs x_i ’s least significant bit.

2 Today's Goal: Encrypting *Multiple* Messages

- **Task:** secret communication of *multiple messages* with shared keys.
- **Threat model:** still computational secrecy against eavesdroppers.
- Two halves of today's plan: (1) a scheme for **multi-message SKE**, and (2) the new technical tool it needs — **how to stretch a PRG** — proved using a brand new proof technique, the **hybrid argument**.

2.1 Setting and the Naive Construction

Setting: Caesar and his general share a key $k \in \{0,1\}^n$ and want to secretly communicate n messages from $\{0,1\}^n$ in the presence of an eavesdropper Eve* (the star denotes that she now gets to see *multiple* ciphertexts, not just one).



Construction idea: stretch the n -bit key k using PRG G into an n^2 -bit output, split it into n pseudorandom one-time pads $y_1, \dots, y_n$ (each n bits), and set $c_i := y_i \oplus m_i$.

★ Added by Claude — Reading the “ $n \mapsto n^2$ ” annotation

The slide's header annotates this with “ $n \mapsto n^2$ ”: the key k is n bits, but G must stretch it to $n \times n = n^2$ bits (enough for n independent n -bit pads $y_1, \dots, y_n$). This is a much bigger stretch than the single-bit stretch ($n \mapsto n+1$) that a “basic” PRG guarantees by definition — which is exactly the technical gap that motivates the “stretching a PRG” section below.

⋈ **Implicit assumption:** the construction is **stateful/synchronous** — both parties must track which message index i they're on. **We lose correctness** if, e.g., ciphertexts are delivered out of order (the receiver would XOR c_2 with y_1 instead of y_2 , recovering garbage).

Exercise 2.1 (Exercise 1, slides). Come up with a scenario in which Eve breaks secrecy using the construction above.

★ Added by Claude — Solution

Suppose (breaking synchrony, or simply due to a retransmission/replay) the *same* pad y_i ends up XORed with two *different* messages — e.g. a dropped-and-resent packet causes $c_i = y_i \oplus m_i$ to be sent twice for two genuinely different plaintexts $m_i \neq m'_i$ (a retransmission where the sender,

out of sync with the receiver, encrypts a *new* message using the pad index the receiver still expects for the *old* one). Eve, observing both ciphertexts $c = y_i \oplus m_i$ and $c' = y_i \oplus m'_i$, computes $c \oplus c' = m_i \oplus m'_i$ — exactly the classic **two-time pad** attack (Lecture 07's central cautionary example, and the same failure mode as IV-reuse in Lecture 11's OFB/CTR modes!). This is precisely *why* the scheme's stateful/synchronous requirement is not a minor implementation footnote, but a security-critical assumption: any desynchronization directly enables a two-time-pad break.

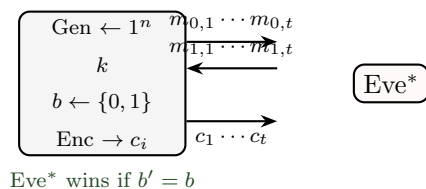
2.2 Formalizing Multi-Message Security

Definition 2.2 (Definition 1, slides — Adversarial Indistinguishability for Multiple Messages). An SKE $\Sigma = (\text{Gen}, \text{Enc}, \text{Dec})$ is **computationally-secret against eavesdroppers who see t messages** if for every (stateful) PPT Eve^* ,

$$\delta(n) := \Pr [\text{Eve}^*(c_1, \dots, c_t) = b] - \frac{1}{2}$$

is negligible, where the probability is over

$((m_{0,1}, \dots, m_{0,t}), (m_{1,1}, \dots, m_{1,t})) \leftarrow \text{Eve}^*(1^n), \quad k \leftarrow \text{Gen}(1^n), \quad b \leftarrow \{0, 1\}, \quad c_i \leftarrow \text{Enc}^*(k, m_{b,i}) \text{ for } i \in [1, t].$



★ Added by Claude — How this compares to Lecture 11's IND-CPA game

This is a natural stepping stone between the single-message eavesdropping game of Lecture 07 and the full IND-CPA game of Lecture 11: Eve^* now submits t *pairs* of messages up front (rather than one pair, or an adaptive stream via an oracle), and must distinguish which entire batch was encrypted. It is still *not* as strong as IND-CPA, because Eve^* commits to all her challenge messages *before* seeing any ciphertexts (no adaptivity, no oracle access) — but it is exactly the right intermediate notion for analyzing schemes, like this lecture's, that must handle more than one message.

Exercise 2.3 (Exercise 2, slides — Assuming synchrony). Prove that if G is a PRG of sufficient stretch, then the construction from the previous slide is secure according to Definition 1.

★ Added by Claude — Solution sketch

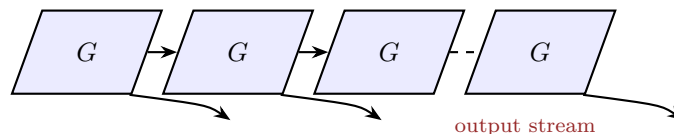
This is a direct reduction to Theorem 1 (Lecture 06/07), *provided* we already know that stretching G to n^2 bits preserves pseudorandomness (exactly the content of Theorem 4, proved in full below!). Given that, the n pads $y_1, \dots, y_n$ are jointly computationally indistinguishable from n independent truly-random pads, so $c_1, \dots, c_n$ are indistinguishable from a genuine one-time-pad encryption of $m_{b,1}, \dots, m_{b,n}$ using *fresh, independent* randomness for each block — and perfect secrecy of the (multi-block) OTP then finishes the argument, exactly as in the single-message case. So really, *this* exercise reduces entirely to the question the rest of the lecture actually

answers: *can we stretch a PRG's output arbitrarily far without losing pseudorandomness?*

3 How to Stretch a PRG

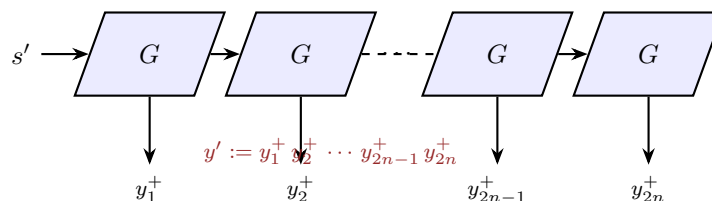
Goal: given a PRG G with stretch $n \mapsto n+1$ (the “basic”, minimal-stretch PRG guaranteed to exist by assumption), construct a PRG G' with stretch $n \mapsto 2n$.

Intuition: think in terms of *die rolls*. Feed a handful of dice into a box G ; out comes one *more* die than you put in, all still looking fair/random. Chain several such boxes together, peeling off one extra “fresh-looking” die at each stage, and after enough stages you’ve turned a small handful of dice into a long row of them.



Construction 3.1 (Construction 3, slides). For a $(n+1)$ -bit string x , write x^- for its n -bit prefix and x^+ for its last bit. Define $G' : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ by iterating G :

- Set $y_1^- \| y_1^+ := G(s')$, i.e. feed the seed s' into G and split its $(n+1)$ -bit output.
- For $j = 2, \dots, 2n$: set $y_j^- \| y_j^+ := G(y_{j-1}^-)$, i.e. feed the previous prefix y_{j-1}^- back into G .
- Output $G'(s') := y' := y_1^+ \| y_2^+ \| \dots \| y_{2n}^+$ (the concatenation of all the “extra bits” peeled off at each stage).



Exercise 3.2 (Exercise 3, slides). Formally write down the construction of G' .

★ Added by Claude — Solution

$G'(s')$:

1. Set $y_0^- := s'$.
2. For $j = 1$ to $2n$: compute y_{j-1}^- 's image $y_j^- \| y_j^+ := G(y_{j-1}^-)$.
3. Output $y' := y_1^+ \| y_2^+ \| \dots \| y_{2n}^+$.

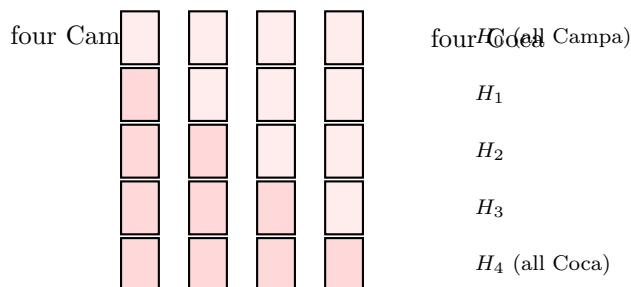
Note G' takes an n -bit seed and makes exactly $2n$ calls to G , each call consuming the n -bit prefix left over from the previous call and producing one new output bit — so G' 's total output length is $2n$ bits, as required. Also note that the final prefix y_{2n}^- is computed but *discarded* (never output) — only the $2n$ “extra” bits $y_1^+, \dots, y_{2n}^+$ make it into $G'(s')$.

4 Proving Pseudorandomness: The Hybrid Argument

4.1 A Warm-Up: The Coca-Cola / Campa-Cola Analogy

Claim 4.1 (Claim 1, slides — informal warm-up). *If you cannot tell apart one can of Campa Cola from one can of Coca Cola, then you cannot tell apart four cans of Campa Cola from four cans of Coca Cola.*

Proof idea: replace the Campa Colas, *one at a time*, with Coca Colas, and argue that no single swap can be detectable (else you’d have a way to tell the two *single* cans apart, contradicting the hypothesis).



4.2 What Is “Coca/Campa Cola” in the PRG Case?

Definition 4.2 (Definition 2, slides — Two-Worlds Definition for Stretch by One Bit). Let G be an efficient deterministic algorithm that for any $n \in \mathbb{N}$ and input $s \in \{0,1\}^n$ outputs a string of length $n+1$. G is a PRG if for every PPT distinguisher D ,

$$\delta(n) := \left| \Pr_{s \leftarrow \{0,1\}^n} [D(G(s)) = 0] - \Pr_{r \leftarrow \{0,1\}^{n+1}} [D(r) = 0] \right|$$

is negligible.

★ Added by Claude — Mapping the analogy

In the cola analogy: a “can of Campa Cola” = a real output $y := G(s)$ of the PRG; a “can of Coca Cola” = a truly random string r of the same length. Claim 1’s informal statement becomes: *if D cannot distinguish one PRG output ($y^- \| y^+ = G(s)$) from one truly random string ($r^- \| r^+$), then D' cannot distinguish $2n$ -many “chained/stretched” pseudorandom bits from $2n$ truly random bits* — this is exactly Theorem 4 below. The “swap one can at a time” proof idea becomes “swap one PRG call’s output for true randomness at a time,” which is exactly the $2n+1$ hybrid worlds $H_0, H_1, \dots, H_{2n}$ constructed in the formal proof.

4.3 Theorem 4, and the Two-Hybrid Warm-Up

Theorem 4.3 (Theorem 4, slides). *If G is a PRG (stretch $n \mapsto n+1$), then so is G' (stretch $n \mapsto 2n$, Construction 3).*

Proof intuition via hybrids (for two iterations). Consider just two chained applications of G , giving three “worlds”:

- H_0 (**pseudorandom world**): $s' \rightarrow G \rightarrow G \rightarrow \text{output } (y_1^+, y_2^+)$ — both extra bits are genuinely produced by chaining G from a random seed.

- H_1 (**hybrid world**): replace the *first* call's output prefix with a truly random string r'_1 , i.e. $s' \rightarrow G$ is thrown away and r'_1 substituted, then $r'_1 \rightarrow G \rightarrow \text{output } (y'_1, y'_2)$ where now $y'_1 := r'_1$ itself and y'_2 still comes from a real G -call on r'_1 .
- H_2 (**random world**): both output bits are replaced by fresh truly-random bits r'_1, r'_2 — nothing here comes from G at all.

Claim 1 (informal). A distinguisher D' for G' with advantage $\geq \delta$ between H_0 and H_2 implies (via an **averaging argument**) there exists some adjacent pair H_i, H_{i+1} , $i \in \{0, 1\}$, that D' distinguishes with advantage $\geq \delta/2$.

Claim 2 (informal). Distinguishing an adjacent pair H_i, H_{i+1} with advantage $\geq \delta/2$ can be turned into a distinguisher D for G itself (with the same advantage), since H_i and H_{i+1} differ *only* in whether one specific G -call's output was real or replaced by true randomness.

Claims 1 and 2 together $\Rightarrow$ Theorem 4.

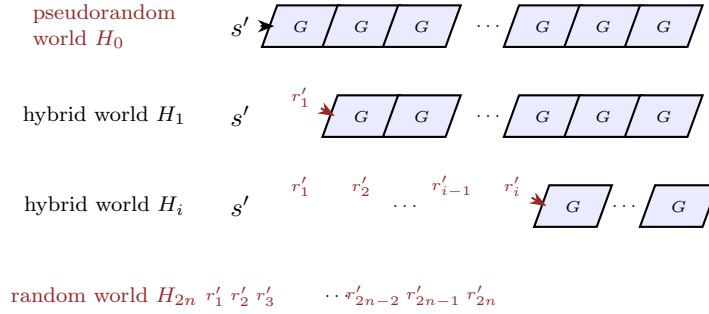
★ Added by Claude — The slides leave the full proof “on whiteboard”

The slide deck itself stops here, noting the full formal proof was worked out live on the whiteboard rather than typed into the slides. The handwritten Lecture 08 notes supply exactly that missing formal proof, generalized from “two iterations” to the full $2n$ -iteration construction. We reproduce it in complete detail below — this is genuinely one of the most technically involved proofs in the course so far, so it's worth working through carefully.

5 The Full Formal Proof of Theorem 4

5.1 Setting Up the $2n + 1$ Hybrid Worlds

Proof. Recall from the lecture that we proceed by a hybrid argument consisting of $2n+1$ worlds $H_0, H_1, \dots, H_{2n-1}, H_{2n}$, illustrated below.



More formally, the hybrid world H_i , $i \in [0, 2n]$, is defined as follows (recall that for an $(n+1)$ -bit string x , x^- denotes its n -bit prefix, while x^+ denotes its last bit):

$H_i(1^n)$:

1. Sample $r'_1, \dots, r'_i \leftarrow \{0, 1\}$.
2. Sample $s' \leftarrow \{0, 1\}^n$, and set $y_i^- := s'$.
3. For $j \in [i+1, 2n]$: compute $y_j^- \| y_j^+ := G(y_{j-1}^-)$; set $y'_{j-1} := y_{j-1}^+$.
4. Output $r'_1, \dots, r'_i, y'_{i+1}, \dots, y'_{2n}$.

★ Added by Claude — Unpacking this definition

H_i 's output has the same length ($2n$ bits) as $G'(s')$ in every hybrid — only its *distribution* changes as i ranges over $0, \dots, 2n$. The *first* i output positions are filled with **truly random** bits $r'_1, \dots, r'_i$ (sampled fresh, with no reference to G at all), while positions $i+1, \dots, 2n$ are filled with the **genuine chained- G** output bits $y'_{i+1}, \dots, y'_{2n}$, exactly as in Construction 3 (just started from a fresh seed s' “relabelled” as y_i^- , i.e. as if the chain had already run i steps).

- H_0 : no positions are random; *all* $2n$ bits come from genuinely chaining G from a real seed s' . This is **exactly** $G'(s')$ — the pseudorandom world.
- H_{2n} : *all* $2n$ positions are random bits $r'_1, \dots, r'_{2n}$, with no G -calls at all. This is a **uniformly random** $2n$ -bit string — the random world.
- H_i for $0 < i < 2n$ interpolates: the first i bits are random, the rest are “real”.

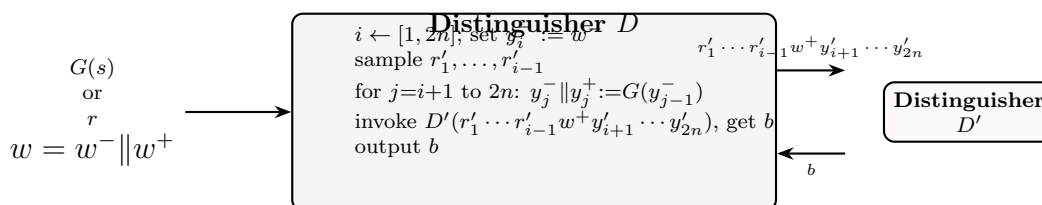
Two adjacent hybrids H_{i-1} and H_i therefore differ in *exactly one* bit's origin — position i switches from being G -chain-derived (in H_{i-1}) to being freshly random (in H_i) — which is precisely the “swap one can at a time” structure from the cola analogy.

5.2 From Adjacent-Hybrid Distinguishing to a PRG Distinguisher

Recall also that we showed in **Claim 1** (using an averaging argument) that if D' can distinguish H_0 from H_{2n} with some probability δ , then there exist two intermediate worlds H_i and H_{i+1} , for some $i \in [0, 2n-1]$, that D' distinguishes with probability at least $\delta/2n$. This distinguishing advantage can, in turn, be exploited to distinguish G 's two worlds, as stated in **Claim 2**. Let's turn this informal argument into a proof.

Assume for contradiction that there exists a distinguisher D' that distinguishes the two worlds of G' . We construct a distinguisher D for the two worlds of G . Recall the reduction from Slide 6 (of the “Theorem 4” section); the analysis follows.

Schematic diagram of the reduction



Pseudocode for $D'(w = w^- || w^+)$

- 1) Sample $i \leftarrow [1, 2n]$ and set $y_i^- := w^-$.
- 2) Sample $r'_1, \dots, r'_{i-1} \leftarrow \{0, 1\}$.
- 3) For $j = i+1$ to $2n$: compute $y_j^- || y_j^+ := G(y_{j-1}^-)$.
- 4) Invoke $D(r'_1 || \dots || r'_{i-1} || w^+ || y'_{i+1} || \dots || y'_{2n})$ to obtain b .
- 5) Output b .

★ Added by Claude — Careful: this is D using D' as a subroutine

Notice the pseudocode header says $D'(w)$ but the box actually describes D 's own algorithm (it invokes D' internally at step 4) — this matches the handwritten notes exactly (they label the pseudocode block “ $D'(w = w^- || w^+)$ ” as shorthand for “here is how D , given w , behaves,” since D 's whole strategy is built around simulating an input to D'). To avoid this ambiguity we've named the box's steps as D 's own algorithm above. In words: D receives w , which is either a genuine PRG output $G(s)$ or a truly random $(n+1)$ -bit string r . D picks a uniformly random “insertion point” $i \in [1, 2n]$, **plants** w at position i in a simulated hybrid (using w^- to continue the G -chain onward, and w^+ as the i -th output bit), fills positions before i with fresh true randomness and positions after i with genuine chained- G output, and hands the resulting $2n$ -bit string to D' . If w is a real PRG output, this simulates exactly hybrid H_{i-1} ; if w is truly random, it simulates exactly hybrid H_i . D then simply **echoes** D' 's guess.

5.3 Analysis

Since D' is assumed to distinguish the two worlds of G' with **non-negligible probability**, there exists a polynomial p , such that for infinitely many n 's,

$$\left| \Pr_{s' \leftarrow \{0,1\}^n} [D'(G'(s')) = 0] - \Pr_{r' \leftarrow \{0,1\}^{2n}} [D'(r') = 0] \right| \geq \frac{1}{p(n)}. \quad (\text{I})$$

By definition of the hybrid worlds, (I) implies that for infinitely-many n 's,

$$\left| \Pr [D'(H_0(1^n)) = 0] - \Pr [D'(H_{2n}(1^n)) = 0] \right| \geq \frac{1}{p(n)}, \quad (\text{II})$$

where the probabilities are over the random coins used in the respective worlds.

Now, let's analyze the distinguisher D . Since D mimics D' 's output, D outputs 0 only if D' does. It follows that

$$\left| \Pr_{s \leftarrow \{0,1\}^n} [D(G(s)) = 0] - \Pr_{r \leftarrow \{0,1\}^{n+1}} [D(r) = 0] \right| = \quad (\text{III})$$

where, writing $y^- \| y^+ := G(s)$,

$$\left| \underbrace{\Pr_{\substack{i \leftarrow [1,2n] \\ r'_1, \dots, r'_{i-1} \leftarrow \{0,1\} \\ s \leftarrow \{0,1\}^n}} [D'(r'_1, \dots, r'_{i-1}, y^+, y'_{i+1}, \dots, y'_{2n}) = 0]}_{= \text{Hybrid world } H_{i-1}} - \underbrace{\Pr_{\substack{i \leftarrow [1,2n] \\ r'_1, \dots, r'_{i-1} \leftarrow \{0,1\} \\ r^-, r^+ \leftarrow \{0,1\}^{n+1}}} [D'(r'_1, \dots, r'_{i-1}, r^+, y'_{i+1}, \dots, y'_{2n}) = 0]}_{= \text{Hybrid world } H_i} \right| \quad (\text{III})$$

Note that for a fixed i , the distribution of $(r'_1, \dots, r'_{i-1}, y^+, y'_{i+1}, \dots, y'_{2n})$ in the top part of (III) is **identical to hybrid** H_{i-1} ; similarly the distribution of $(r'_1, \dots, r'_{i-1}, r^+, y'_{i+1}, \dots, y'_{2n})$ in its bottom part is **identical to** H_i . Therefore

$$\begin{aligned} (\text{III}) &= \left| \Pr_{i \leftarrow [1,2n]} [D'(H_{i-1}) = 0] - \Pr_{i \leftarrow [1,2n]} [D'(H_i) = 0] \right| \\ &= \frac{1}{2n} \left| \sum_{i=1}^{2n} \left(\Pr [D'(H_{i-1}) = 0] - \Pr [D'(H_i) = 0] \right) \right| \quad (\text{telescoping sum}) \\ &= \frac{1}{2n} \left| \Pr [D'(H_0) = 0] - \Pr [D'(H_{2n}) = 0] \right| \\ &\stackrel{(\text{II})}{\geq} \frac{1}{2n} \cdot \frac{1}{p(n)}. \end{aligned}$$

Thus, putting everything together, we get that for infinitely many n 's,

$$\left| \Pr_{s \leftarrow \{0,1\}^n} [D(G(s)) = 0] - \Pr_{r \leftarrow \{0,1\}^{n+1}} [D(r) = 0] \right| \geq \frac{1}{2n p(n)}. \quad (\text{"security loss"})$$

However, this contradicts G 's pseudorandomness. ■

★ Added by Claude — Walking through the telescoping-sum step

The step from the sum of $2n$ differences down to just $\Pr[D'(H_0)=0] - \Pr[D'(H_{2n})=0]$ is a classic **telescoping sum**: writing out the terms,

$$\begin{aligned} \sum_{i=1}^{2n} (\Pr[D'(H_{i-1})=0] - \Pr[D'(H_i)=0]) &= (\Pr[D'(H_0)=0] - \Pr[D'(H_1)=0]) \\ &\quad + (\Pr[D'(H_1)=0] - \Pr[D'(H_2)=0]) + \cdots \\ &\quad + (\Pr[D'(H_{2n-1})=0] - \Pr[D'(H_{2n})=0]), \end{aligned}$$

every intermediate term $\Pr[D'(H_i)=0]$ (for $0 < i < 2n$) appears once with a $+$ and once with a $-$, canceling out, leaving only the very first and very last terms, $\Pr[D'(H_0)=0] - \Pr[D'(H_{2n})=0]$. This is the rigorous version of the informal “averaging argument” (Claim 1) from the two-iteration warm-up: rather than arguing *some* adjacent pair must have advantage $\geq \delta/2n$ and then separately building a reduction for that unspecified pair, the formal proof folds *all* $2n$ adjacent-pair comparisons into a single random choice of $i \in [1, 2n]$ inside D itself, and the telescoping sum shows this single random- i strategy’s expected/average distinguishing power is *exactly* $\frac{1}{2n}$ times the full H_0 -vs- H_{2n} gap — no case analysis on “which i ” is needed.

6 Taking Stock: The Cost of a Hybrid Argument

Construction 3 and Theorem 4 work for any polynomial stretch (chaining G a polynomial number of times). **What happens if we stretch it *exponentially*?**

★ Added by Claude — Why exponential stretch breaks the argument

The security-loss bound we derived, $\frac{1}{2n p(n)}$, has $2n$ (the number of hybrid steps) in the *denominator*. The hybrid argument still goes through mathematically for exponentially many hybrids, but the resulting bound $\frac{1}{2^{\Theta(n)} p(n)}$ is not usefully bounded away from zero relative to what “negligible” requires you to rule out — worse, D ’s own running time typically must scale with the number of hybrids simulated (it needs to *run* the G -chain up to length i for a random i), so an exponential number of hybrids would make D itself take exponential time, which disqualifies it as a valid PPT PRG-distinguisher in the first place. This is exactly why PRG length-extension constructions in the literature are always stated for *polynomial* stretch only.

There is also a $2n$ “**loss in pseudorandomness**”: if D' distinguishes with some probability $1/p(n)$, then D distinguishes with probability only $\approx \frac{1}{2n p(n)}$ — the more the stretch, the greater the loss. More generally, this is called the **loss in security** of a reduction: one way to measure how “wasteful” the reduction is.

Exercise 6.1 (Exercise 4, slides). Think of a less wasteful reduction strategy for Theorem 4. Do you feel it is possible? Maybe you need a different construction?

★ Added by Claude — Some genuine food for thought

This is a deep and, in general, **open-ended** question — there is no “the” answer, but here are pointers worth thinking about (and the slides’ own “Further Material” section flags this general topic, citing Chatterjee, Menezes, and Sarkar’s “Another Look at Tightness”):

- **Tightness is construction-dependent.** The $O(n)$ loss here comes specifically from the *sequential-chaining* structure of Construction 3 (each hybrid differs from its neighbor by exactly one G -call). A fundamentally different length-extension construction — e.g. a **tree-based (GGM-style) construction**, where G ’s output is used to derive *two* independent sub-seeds at each level, doubling the output length at every step of a binary tree of depth $\log n$ rather than linearly chaining $2n$ times — can sometimes reduce the number of *hybrids* needed from $O(n)$ to $O(\log n)$, at the cost of a more complex construction and proof.
- **Some loss may be inherent.** For certain primitives and reduction styles, researchers have proven *lower bounds* showing that a black-box reduction of this general shape *cannot* avoid a loss related to the number of hybrid steps — i.e. the looseness isn’t just a failure of cleverness, but a genuine limitation of this proof technique applied in a black-box way. This is an active research area (“**tight reductions**” in cryptography) precisely because, as the next box explains, loss has real practical consequences.

6.1 Why Loss in Security Matters

To be safe, G' must be instantiated at a **larger security level** than G . Suppose a distinguisher D was found for G that runs in n^4 steps and succeeds with probability at most $1/n^2$.

Effect of D on G	Effect of D on G'
$n = 50$: D working for ≈ 3 months, breaks with probability $\approx 1/2500$ — acceptable .	$n = 50$: cannot rule out D' working for ≈ 3 months, breaking with probability $\approx 1/50$ — breakable!
$n = 100$: D working for ≈ 3 years, breaks with probability 2^{-200} — safe .	$n = 100$: D' working for ≈ 3 years, breaking with probability $\approx 2^{-100}$ — safe .

★ Added by Claude — The headline lesson

At $n = 50$, a loss factor of $2n = 100$ turns an *acceptable* $1/2500$ break-probability against G into a genuinely *dangerous* $\approx 1/50$ break-probability against G' — the reduction's looseness has directly degraded real-world security at that parameter size. At $n = 100$, the same $2n = 200\times$ loss factor is comparatively harmless, since 2^{-200} divided by 200 is still astronomically small ($\approx 2^{-100}$, still “safe”). **The practical takeaway:** when your security proof has a loss factor of L , you must pick your security parameter n large enough that even after dividing your target security level by L , you're still comfortably safe — in other words, *loose reductions cost you extra key length/computation in the real world*, even though the scheme is, strictly speaking, still “provably secure” at every parameter size. This is precisely why cryptographers care about tight reductions: a tighter proof lets you use *smaller*, faster parameters for the same real-world security guarantee.

7 Recap and Next Lecture

To recap:

- Learnt how to encrypt *multiple* messages, but only with **state/synchrony**.
- This boiled down to **stretching a PRG** to arbitrary polynomial length.
- **Key tool:** the **hybrid argument** — swap one “can” (one G -call’s output) at a time between the pseudorandom and truly-random worlds, and use a telescoping sum / averaging argument to turn a full end-to-end distinguisher into a reduction against a single step.

Next lecture: how to **avoid** state/synchrony:

- Using a new primitive: the **pseudorandom function (PRF)**.
- How to construct a PRF from a PRG.
- A stronger attack model: **chosen-plaintext attack (CPA)**.
- CPA-secure SKE from a PRF.

★ Added by Claude — This is exactly where Lecture 11 picks up!

This “next lecture” preview is precisely the origin story of the PRF-based scheme (Construction 10.1) whose IND-CPA security proof you already worked through in the Lecture 11 notes: a PRF lets you compute $F_k(i)$ for *any* index i **on demand**, statelessly and out of order (unlike this lecture’s stretched-PRG approach, which needs both parties to track a shared position in a single long, sequentially-generated pseudorandom stream) — directly solving the synchrony problem raised in this lecture’s Exercise 1. It’s a satisfying throughline: Lecture 07 built a PRG and proved single-message security; Lecture 08 (this lecture) tried the “obvious” multi-message extension (stretch the PRG) and hit a real practical wall (synchrony); Lectures 9-10 (not covered in the notes you’ve uploaded so far) introduce the PRF as the fix; and Lecture 11 proves the resulting scheme fully IND-CPA-secure — reusing, almost verbatim, this very lecture’s OTP-style challenge-query distinguishing strategy (itself inherited from Lecture 07’s Theorem 1) inside its own reduction.

Further Material (as noted in the slides)

- For more on the factoring-based stream cipher (Blum–Blum–Shub), the original paper [1] is an excellent source: it shows how the reduction algorithm works — given an adversary that predicts the stream cipher, one can decide the **quadratic residuosity problem**, closely related to factoring integers.
- For more on the effect of loss in security reductions (directly relevant to this lecture’s Exercise 4 and the discussion above), see Chatterjee, Menezes, and Sarkar, “Another Look at Tightness” [2].

Sources (as cited in the slides)

1. Security for multiple encryptions: §3.4.1 of Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed.), 2014.

2. Stateful encryption: §3.1.1 of the same reference.
3. PRG stretching: §8.4.2 of the same reference.

References

- [1] L. Blum, M. Blum, and M. Shub. *A simple unpredictable pseudo-random number generator*. SIAM Journal on Computing, 15(2):364–383, 1986.
- [2] Sanjit Chatterjee, Alfred Menezes, and Palash Sarkar. *Another look at tightness*. In SAC 2011, volume 7118 of LNCS, pages 293–319. Springer, Berlin, Heidelberg, August 2012.

★ Added by Claude — Quick self-check questions

1. Why does the naive multi-message construction lose *correctness* (not just security) if ciphertexts arrive out of order?
2. In hybrid world H_i , which output positions are truly random and which come from chaining G ? What does H_0 collapse to? What does H_{2n} collapse to?
3. In the formal reduction, why does D sample a *uniformly random* insertion point $i \in [1, 2n]$, rather than, say, always using $i = 1$?
4. Walk through, in your own words, why the telescoping sum in the Analysis section collapses to just the H_0 -vs- H_{2n} gap divided by $2n$.
5. Why does a $2n\times$ loss in a security reduction matter more at small security parameters ($n = 50$) than at large ones ($n = 100$)?
6. How does using a PRF (previewed for next lecture) sidestep the synchrony problem that motivated this entire lecture's PRG-stretching detour?